

```

In [1]: # =====
# CREDIT CARD FRAUD DETECTION PROJECT
# USING:
# 1. Logistic Regression
# 2. Random Forest
# 3. XGBoost
# 4. Isolation Forest
# 5. Autoencoder
# =====

# =====
# 1. IMPORT LIBRARIES
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import IsolationForest

from xgboost import XGBClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score,
    roc_curve
)

from imblearn.over_sampling import SMOTE

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.callbacks import EarlyStopping

import warnings
warnings.filterwarnings("ignore")

```

```

In [2]: # =====
# 2. LOAD DATASET
# =====

df = pd.read_csv("creditcard.csv")

print(df.head())

print("\nDataset Shape:")
print(df.shape)

```

	Time	V1	V2	V3	V4	V5	V6	V7	\
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	

	V8	V9	...	V21	V22	V23	V24	V25	\
0	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	
1	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	
2	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	
3	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	
4	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	

	V26	V27	V28	Amount	Class
0	-0.189115	0.133558	-0.021053	149.62	0
1	0.125895	-0.008983	0.014724	2.69	0
2	-0.139097	-0.055353	-0.059752	378.66	0
3	-0.221929	0.062723	0.061458	123.50	0
4	0.502292	0.219422	0.215153	69.99	0

[5 rows x 31 columns]

Dataset Shape:
(284807, 31)

```
In [3]: # =====
# 3. EXPLORATORY DATA ANALYSIS (EDA)
# =====

print("\nDataset Information:")
print(df.info())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nClass Distribution:")
print(df['Class'].value_counts())

# Fraud vs Normal Plot
plt.figure(figsize=(6,4))
sns.countplot(x='Class', data=df)
plt.title("Fraud vs Normal Transactions")
plt.show()

# Amount Distribution
plt.figure(figsize=(8,4))
sns.histplot(df['Amount'], bins=50)
plt.title("Amount Distribution")
plt.show()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Time        284807 non-null  float64
1   V1          284807 non-null  float64
2   V2          284807 non-null  float64
3   V3          284807 non-null  float64
4   V4          284807 non-null  float64
5   V5          284807 non-null  float64
6   V6          284807 non-null  float64
7   V7          284807 non-null  float64
8   V8          284807 non-null  float64
9   V9          284807 non-null  float64
10  V10         284807 non-null  float64
11  V11         284807 non-null  float64
12  V12         284807 non-null  float64
13  V13         284807 non-null  float64
14  V14         284807 non-null  float64
15  V15         284807 non-null  float64
16  V16         284807 non-null  float64
17  V17         284807 non-null  float64
18  V18         284807 non-null  float64
19  V19         284807 non-null  float64
20  V20         284807 non-null  float64
21  V21         284807 non-null  float64
22  V22         284807 non-null  float64
23  V23         284807 non-null  float64
24  V24         284807 non-null  float64
25  V25         284807 non-null  float64
26  V26         284807 non-null  float64
27  V27         284807 non-null  float64
28  V28         284807 non-null  float64
29  Amount      284807 non-null  float64
30  Class       284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
None
```

Missing Values:

```
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
```

```
V24      0
V25      0
V26      0
V27      0
V28      0
Amount    0
Class     0
dtype: int64
```

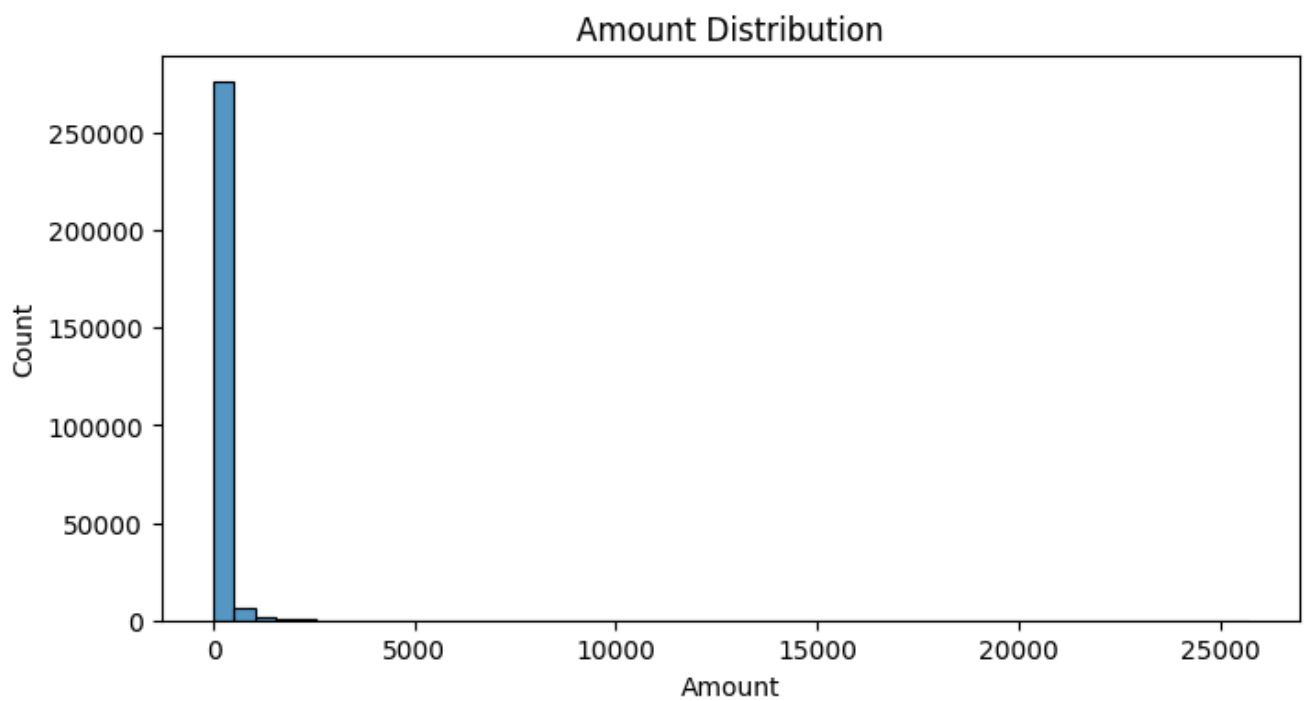
Class Distribution:

Class

```
0      284315
```

```
1         492
```

Name: count, dtype: int64



```
In [4]: # =====
# 4. DATA PREPROCESSING
# =====

scaler = StandardScaler()
```

```

df['scaled_amount'] = scaler.fit_transform(df[['Amount']])
df['scaled_time'] = scaler.fit_transform(df[['Time']])

df.drop(['Amount', 'Time'], axis=1, inplace=True)

X = df.drop('Class', axis=1)
y = df['Class']

```

```

In [5]: # =====
# 5. FEATURE SELECTION
# =====

temp_rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

temp_rf.fit(X, y)

importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': temp_rf.feature_importances_
})

importance_df = importance_df.sort_values(
    by='Importance',
    ascending=False
)

print("\nTop 10 Important Features:")
print(importance_df.head(10))

```

Top 10 Important Features:

	Feature	Importance
16	V17	0.190009
13	V14	0.130959
11	V12	0.107772
15	V16	0.073048
9	V10	0.071184
10	V11	0.062644
3	V4	0.030328
8	V9	0.029658
6	V7	0.027435
17	V18	0.023388

```

In [6]: # =====
# 6. SPLIT THE DATASET
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)

```

Training Shape: (227845, 30)

Testing Shape: (56962, 30)

```

In [7]: # =====
# 7. HANDLE IMBALANCED DATA USING SMOTE
# =====

```

```

smote = SMOTE(random_state=42)

X_train_smote, y_train_smote = smote.fit_resample(
    X_train,
    y_train
)

print("\nAfter SMOTE:")
print(y_train_smote.value_counts())

```

After SMOTE:

Class

0 227451

1 227451

Name: count, dtype: int64

```

In [8]: # =====
# 8. LOGISTIC REGRESSION
# =====

lr = LogisticRegression(max_iter=1000)

lr.fit(X_train_smote, y_train_smote)

y_pred_lr = lr.predict(X_test)

```

```

In [9]: # =====
# 9. RANDOM FOREST
# =====

rf = RandomForestClassifier(
    n_estimators=200,
    random_state=42
)

rf.fit(X_train_smote, y_train_smote)

y_pred_rf = rf.predict(X_test)

```

```

In [10]: # =====
# 10. XGBOOST
# =====

xgb = XGBClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=6,
    random_state=42,
    eval_metric='logloss'
)

xgb.fit(X_train_smote, y_train_smote)

y_pred_xgb = xgb.predict(X_test)

```

```

In [11]: # =====
# 11. ISOLATION FOREST
# =====

iso = IsolationForest(
    n_estimators=100,
    contamination=0.001,
    random_state=42
)

```

```

iso.fit(X_train)

y_pred_iso = iso.predict(X_test)

# Convert output
y_pred_iso = np.where(y_pred_iso == -1, 1, 0)

```

```

In [12]: # =====
# 12. AUTOENCODER
# =====

# Train only on normal transactions
X_train_normal = X_train[y_train == 0]

input_dim = X_train.shape[1]

input_layer = Input(shape=(input_dim,))

encoder = Dense(16, activation='relu')(input_layer)
encoder = Dense(8, activation='relu')(encoder)

decoder = Dense(16, activation='relu')(encoder)
decoder = Dense(input_dim, activation='linear')(decoder)

autoencoder = Model(inputs=input_layer, outputs=decoder)

autoencoder.compile(
    optimizer='adam',
    loss='mse'
)

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

history = autoencoder.fit(
    X_train_normal,
    X_train_normal,
    epochs=20,
    batch_size=256,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)

# Reconstruction
reconstruction = autoencoder.predict(X_test)

mse = np.mean(
    np.power(X_test - reconstruction, 2),
    axis=1
)

threshold = np.percentile(mse, 95)

y_pred_auto = np.where(mse > threshold, 1, 0)

```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Epoch 1/20

711/711  6s 5ms/step - loss: 0.6736 - val_loss: 0.4630

Epoch 2/20

711/711  5s 4ms/step - loss: 0.4181 - val_loss: 0.3879

Epoch 3/20

711/711  3s 4ms/step - loss: 0.3697 - val_loss: 0.3524

Epoch 4/20

711/711  5s 4ms/step - loss: 0.3404 - val_loss: 0.3301

Epoch 5/20

711/711  5s 4ms/step - loss: 0.3241 - val_loss: 0.3183

Epoch 6/20

711/711  3s 4ms/step - loss: 0.3141 - val_loss: 0.3091

Epoch 7/20

711/711  6s 4ms/step - loss: 0.3064 - val_loss: 0.3031

Epoch 8/20

711/711  3s 4ms/step - loss: 0.2992 - val_loss: 0.2960

Epoch 9/20

711/711  5s 4ms/step - loss: 0.2935 - val_loss: 0.2901

Epoch 10/20

711/711  5s 4ms/step - loss: 0.2872 - val_loss: 0.2847

Epoch 11/20

711/711  5s 4ms/step - loss: 0.2828 - val_loss: 0.2884

Epoch 12/20

711/711  5s 4ms/step - loss: 0.2789 - val_loss: 0.2766

Epoch 13/20

711/711  5s 4ms/step - loss: 0.2752 - val_loss: 0.2741

Epoch 14/20

711/711  5s 4ms/step - loss: 0.2718 - val_loss: 0.2698

Epoch 15/20

711/711  5s 4ms/step - loss: 0.2685 - val_loss: 0.2664

Epoch 16/20

711/711  3s 4ms/step - loss: 0.2653 - val_loss: 0.2638

Epoch 17/20

711/711  3s 4ms/step - loss: 0.2627 - val_loss: 0.2612

Epoch 18/20

711/711  5s 4ms/step - loss: 0.2605 - val_loss: 0.2601

Epoch 19/20

711/711  3s 4ms/step - loss: 0.2592 - val_loss: 0.2582

Epoch 20/20

711/711  5s 4ms/step - loss: 0.2580 - val_loss: 0.2579

1781/1781  3s 2ms/step

```
In [13]: # =====
# 13. MODEL EVALUATION FUNCTION
# =====

def evaluate_model(name, y_true, y_pred):

    print("\n=====")
    print(f"{name} Results")
    print("=====")

    accuracy = accuracy_score(y_true, y_pred)

    precision = precision_score(y_true, y_pred)

    recall = recall_score(y_true, y_pred)

    f1 = f1_score(y_true, y_pred)

    roc_auc = roc_auc_score(y_true, y_pred)

    print("Accuracy :", accuracy)
```

```

print("Precision:", precision)
print("Recall    :", recall)
print("F1 Score :", f1)
print("ROC AUC   :", roc_auc)

print("\nClassification Report:")
print(classification_report(y_true, y_pred))

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(5,4))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.title(f"{name} Confusion Matrix")

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()

return {
    "Model": name,
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1 Score": f1,
    "ROC AUC": roc_auc
}

```

```

In [14]: # =====
# 14. EVALUATE ALL MODELS
# =====

results = []

results.append(
    evaluate_model(
        "Logistic Regression",
        y_test,
        y_pred_lr
    )
)

results.append(
    evaluate_model(
        "Random Forest",
        y_test,
        y_pred_rf
    )
)

results.append(
    evaluate_model(
        "XGBoost",
        y_test,
        y_pred_xgb
    )
)

```

```

results.append(
    evaluate_model(
        "Isolation Forest",
        y_test,
        y_pred_iso
    )
)

results.append(
    evaluate_model(
        "Autoencoder",
        y_test,
        y_pred_auto
    )
)

```

```

=====
Logistic Regression Results
=====

```

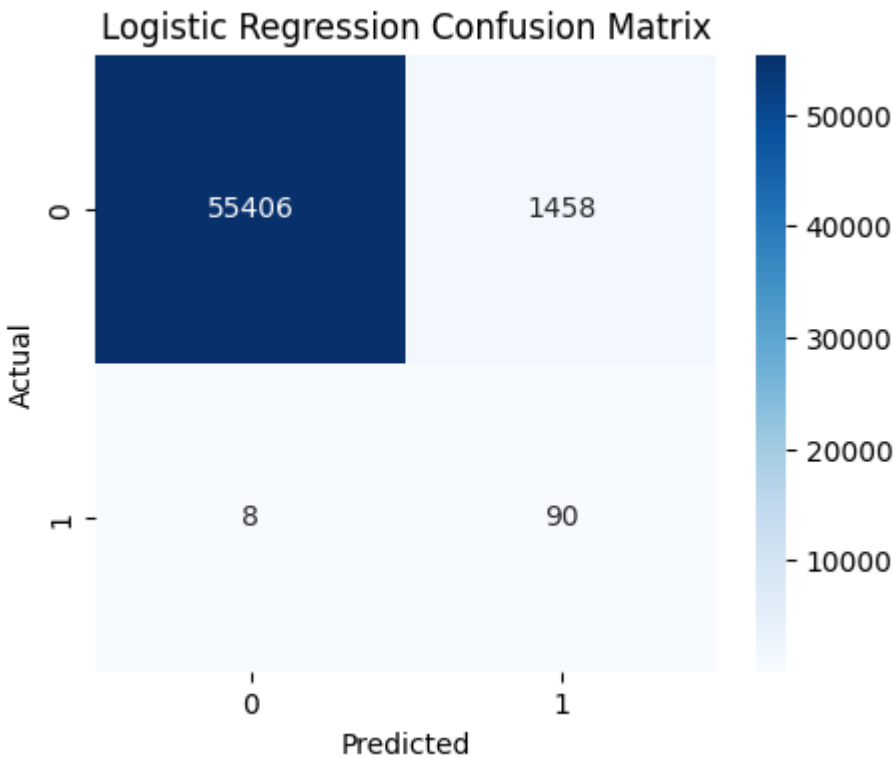
```

Accuracy : 0.9742635441171307
Precision: 0.05813953488372093
Recall   : 0.9183673469387755
F1 Score : 0.10935601458080195
ROC AUC  : 0.9463636115673055

```

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.97	0.99	56864
1	0.06	0.92	0.11	98
accuracy			0.97	56962
macro avg	0.53	0.95	0.55	56962
weighted avg	1.00	0.97	0.99	56962

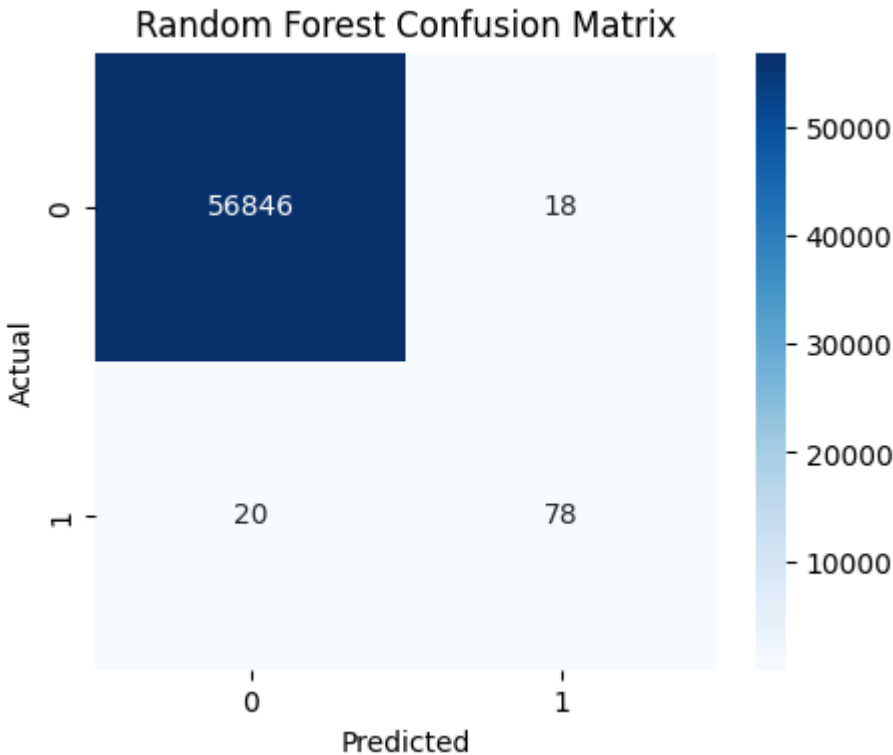


Random Forest Results

Accuracy : 0.9993328885923949
Precision: 0.8125
Recall : 0.7959183673469388
F1 Score : 0.8041237113402062
ROC AUC : 0.8978009113043078

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.81	0.80	0.80	98
accuracy			1.00	56962
macro avg	0.91	0.90	0.90	56962
weighted avg	1.00	1.00	1.00	56962



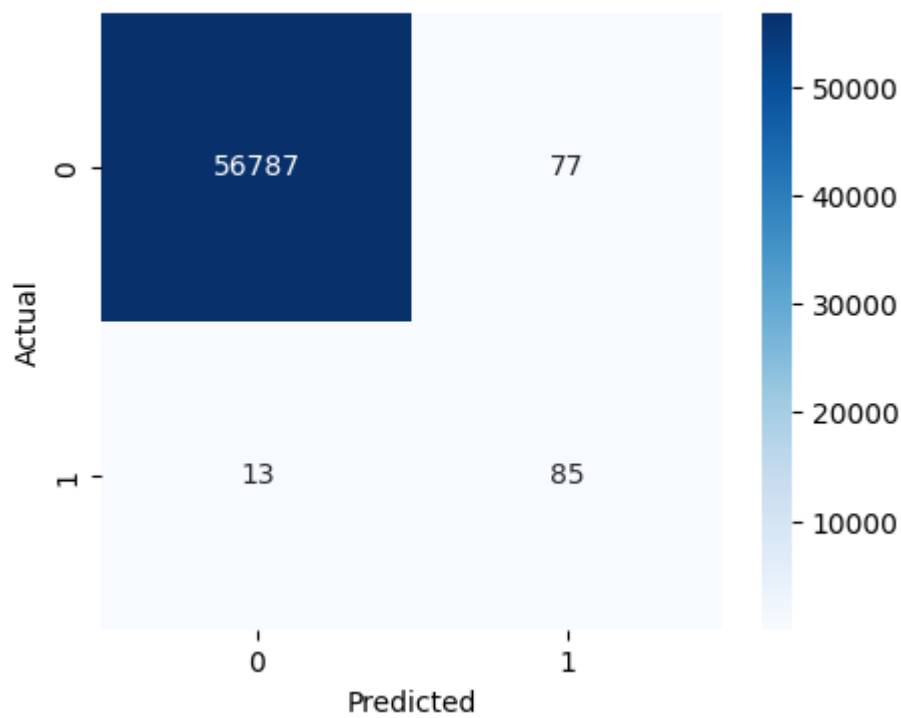
XGBoost Results

Accuracy : 0.9984199992977775
Precision: 0.5246913580246914
Recall : 0.8673469387755102
F1 Score : 0.6538461538461539
ROC AUC : 0.9329964153641198

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.52	0.87	0.65	98
accuracy			1.00	56962
macro avg	0.76	0.93	0.83	56962
weighted avg	1.00	1.00	1.00	56962

XGBoost Confusion Matrix



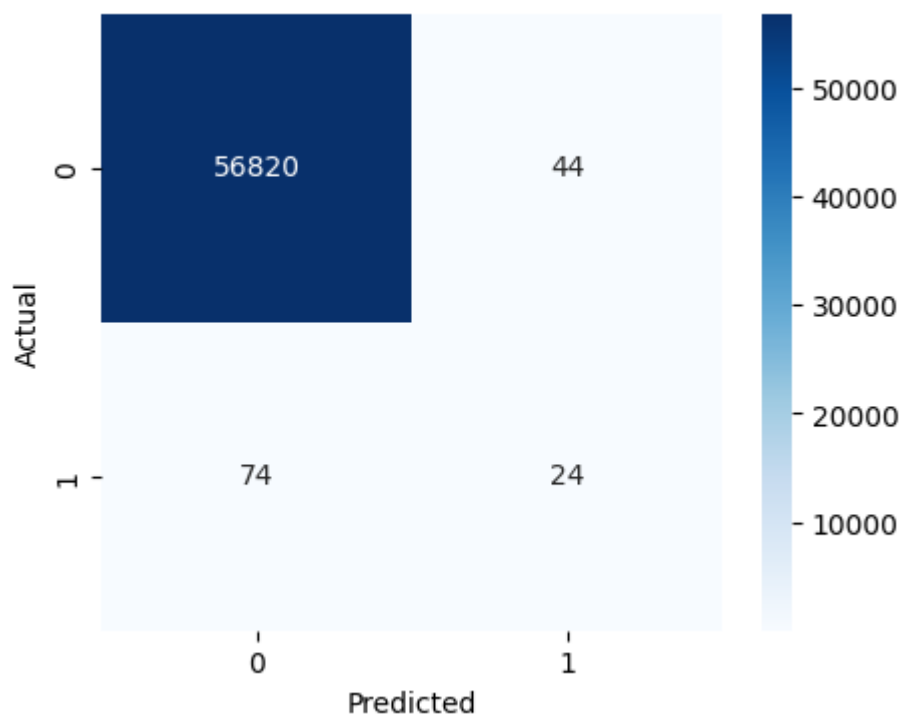
=====
Isolation Forest Results
=====

Accuracy : 0.9979284435237527
Precision: 0.35294117647058826
Recall : 0.24489795918367346
F1 Score : 0.2891566265060241
ROC AUC : 0.6220620915783309

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	56864
1	0.35	0.24	0.29	98
accuracy			1.00	56962
macro avg	0.68	0.62	0.64	56962
weighted avg	1.00	1.00	1.00	56962

Isolation Forest Confusion Matrix



Autoencoder Results

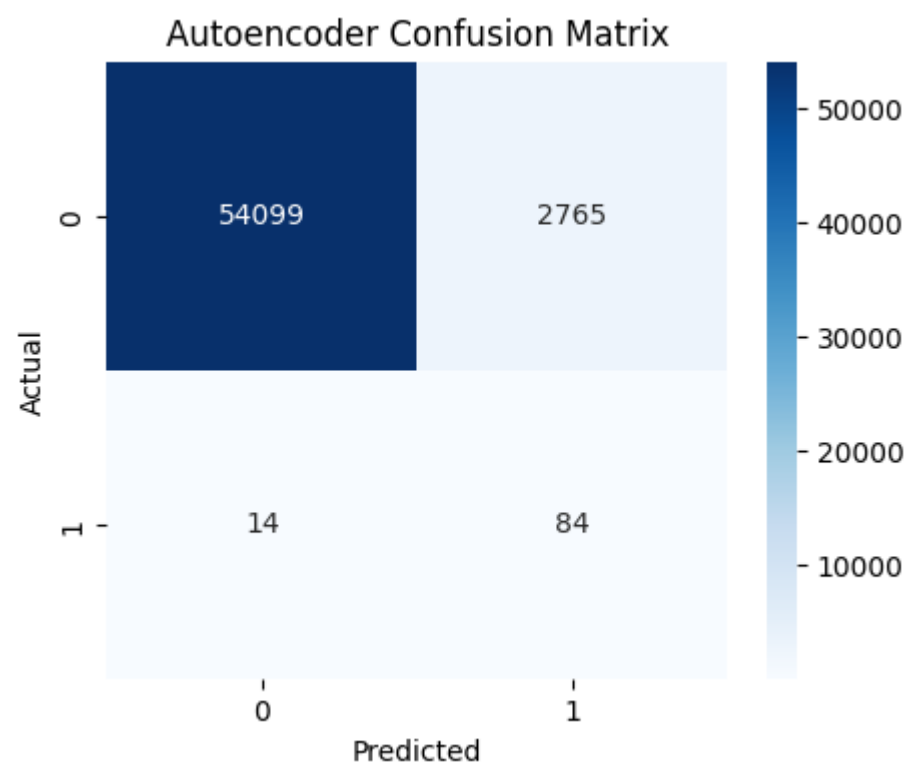
```

=====
Accuracy : 0.9512130894280397
Precision: 0.029484029484029485
Recall   : 0.8571428571428571
F1 Score : 0.057007125890736345
ROC AUC  : 0.9042590340863413

```

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.95	0.97	56864
1	0.03	0.86	0.06	98
accuracy			0.95	56962
macro avg	0.51	0.90	0.52	56962
weighted avg	1.00	0.95	0.97	56962



```

In [15]: # =====
# 15. FINAL MODEL COMPARISON
# =====

results_df = pd.DataFrame(results)

print("\nFinal Comparison Table:")
print(results_df)

```

Final Comparison Table:

	Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
0	Logistic Regression	0.974264	0.058140	0.918367	0.109356	0.946364
1	Random Forest	0.999333	0.812500	0.795918	0.804124	0.897801
2	XGBoost	0.998420	0.524691	0.867347	0.653846	0.932996
3	Isolation Forest	0.997928	0.352941	0.244898	0.289157	0.622062
4	Autoencoder	0.951213	0.029484	0.857143	0.057007	0.904259

```

In [16]: # =====
# 16. ROC CURVE
# =====

plt.figure(figsize=(8,6))

```

```

# Logistic Regression
lr_probs = lr.predict_proba(X_test)[: ,1]
fpr, tpr, _ = roc_curve(y_test, lr_probs)
plt.plot(fpr, tpr, label='Logistic Regression')

# Random Forest
rf_probs = rf.predict_proba(X_test)[: ,1]
fpr, tpr, _ = roc_curve(y_test, rf_probs)
plt.plot(fpr, tpr, label='Random Forest')

# XGBoost
xgb_probs = xgb.predict_proba(X_test)[: ,1]
fpr, tpr, _ = roc_curve(y_test, xgb_probs)
plt.plot(fpr, tpr, label='XGBoost')

plt.plot([0,1], [0,1], linestyle='--')

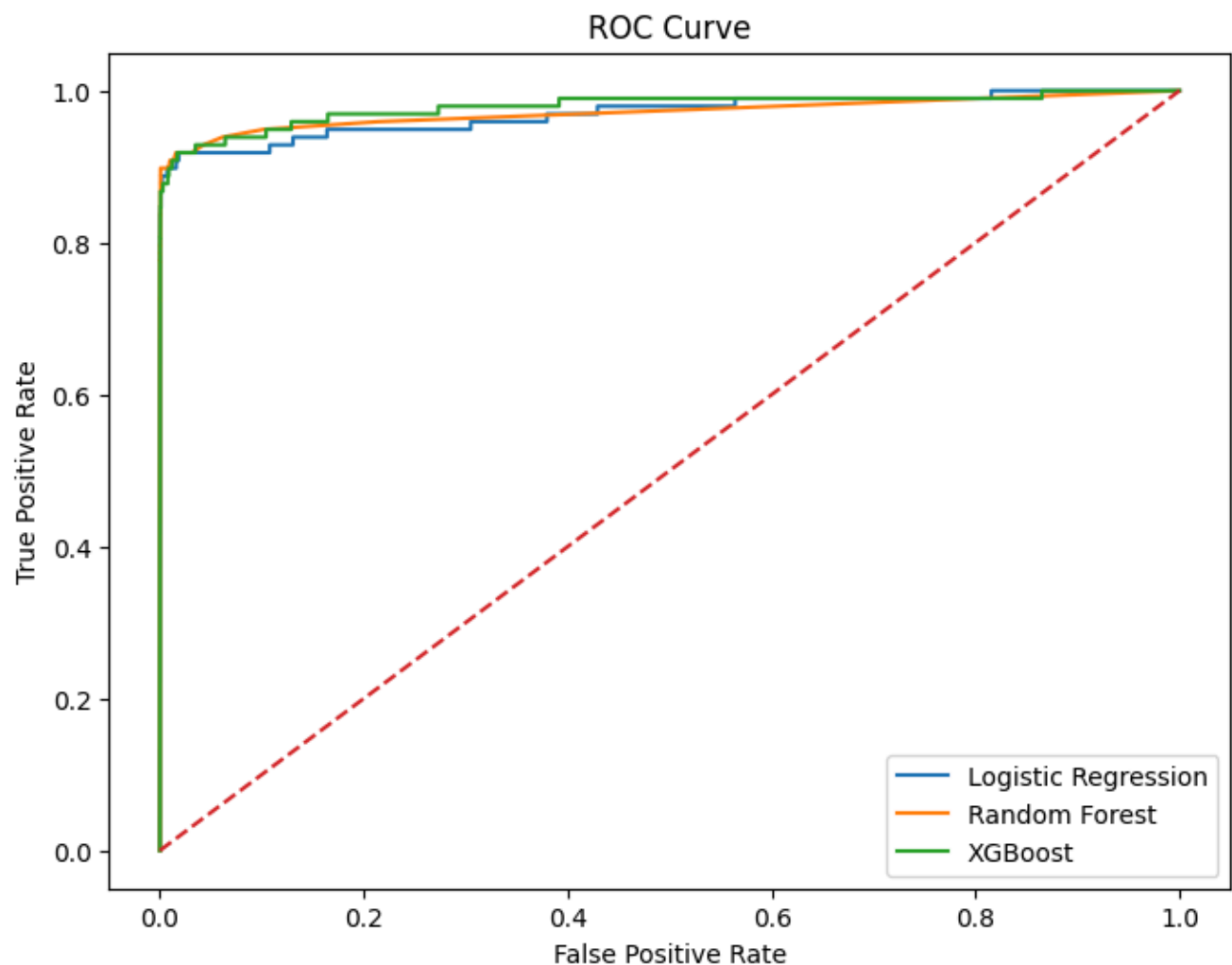
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

plt.title("ROC Curve")

plt.legend()

plt.show()

```



```

In [17]: # =====
# 17. HYPERPARAMETER TUNING
# =====

from sklearn.model_selection import RandomizedSearchCV

param_grid = {
    'n_estimators': [100, 200],

```

```

    'max_depth': [4, 6, 8],
    'learning_rate': [0.01, 0.05, 0.1]
}

random_search = RandomizedSearchCV(
    estimator=XGBClassifier(eval_metric='logloss'),
    param_distributions=param_grid,
    n_iter=5,
    scoring='f1',
    cv=3,
    verbose=1,
    random_state=42
)

random_search.fit(
    X_train_smote,
    y_train_smote
)

print("\nBest Parameters:")
print(random_search.best_params_)

```

Fitting 3 folds for each of 5 candidates, totalling 15 fits

Best Parameters:

```
{'n_estimators': 200, 'max_depth': 8, 'learning_rate': 0.01}
```

```

In [18]: from sklearn.model_selection import train_test_split
import pandas as pd

X = df.drop('Class', axis=1)
y = df['Class']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

In [19]: # =====
# SAMPLE PREDICTIONS
# =====

# Take first 10 transactions from test data
sample_data = X_test.iloc[:10]

# Actual values
actual_values = y_test.iloc[:10].values

# Predict using Random Forest
predictions = rf.predict(sample_data)

# Create result table
sample_results = pd.DataFrame({
    "Actual Class": actual_values,
    "Predicted Class": predictions
})

# Convert values to Labels
sample_results["Actual Label"] = sample_results["Actual Class"].map({
    0: "Legitimate",
    1: "Fraud"
})

```

```
sample_results["Predicted Label"] = sample_results["Predicted Class"].map({
    0: "Legitimate",
    1: "Fraud"
})

print("\nSample Predictions:")
print(sample_results)
```

Sample Predictions:

	Actual Class	Predicted Class	Actual Label	Predicted Label
0	0	0	Legitimate	Legitimate
1	0	0	Legitimate	Legitimate
2	0	0	Legitimate	Legitimate
3	0	0	Legitimate	Legitimate
4	0	0	Legitimate	Legitimate
5	0	0	Legitimate	Legitimate
6	0	0	Legitimate	Legitimate
7	0	0	Legitimate	Legitimate
8	0	0	Legitimate	Legitimate
9	0	0	Legitimate	Legitimate

In []: